

计算机视觉第一次作业——硬币定位系统

一、作业要求与完成情况

1. 作业要求

编写一个钱币定位系统，不仅能够检测出输入图像中各个钱币的边缘，同时给出各个钱币的圆心坐标与半径。可以直接调用 OpenCV 的 Canny 与 HoughCircle 算法，也可以自行实现这些算法。

2. 完成情况

本次作业完成了以下工作：

- 整体框架设计：**设计并实现了一套灵活的 benchmark 框架，采用策略模式可快速切换不同的算法实现，方便比对 OpenCV 和自实现算法的性能差异。
- Canny 边缘检测：**使用 C++实现了高性能 Canny 边缘检测算法，采用多种优化策略，并通过 pybind11 将其包装为 Python 模块，实现了接近 OpenCV 的边缘提取性能（平均像素差异：0，时间差异 $\approx$ 20ms）。
- HoughCircle 圆形检测：**在 Python 中实现了 HoughCircle 算法，可准确检测图像中的圆形。
- Web 调参应用：**基于 Flask 框架开发了一个交互式 Web 应用，支持实时调整算法参数并可视化结果，大大提高了参数调优的效率。

二、算法流程与原理

1. Canny 边缘检测算法

Canny 边缘检测算法是一种经典的边缘检测方法，其基本流程包含几个步骤。首先，使用高斯核对图像进行平滑处理，减少噪声影响。之后，使用 Sobel 算子计算图像的梯度幅值和方向，并沿梯度方向比较，仅保留局部最大值。最后，使用高低两个阈值确定强边缘和弱边缘，将与强边缘相连的弱边缘保留，其余弱边缘丢弃，就获得了图像边缘。

2. Hough 变换圆检测算法

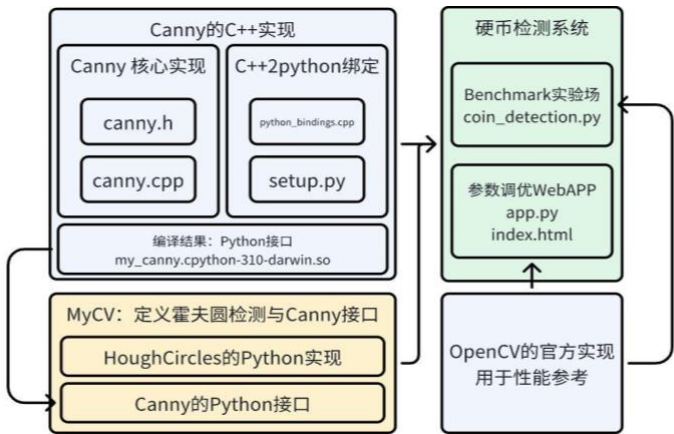
Hough 变换是一种用于检测规则几何形状的算法，其检测圆形的原理如下。首先，我们要构建参数空间，圆可以用参数方程 $x = a + r \cdot \cos(\theta)$ ， $y = b + r \cdot \sin(\theta)$ 表示，其中(a,b)是圆心，r 是半径，这些参数可以张成一个空间。之后，为每个可能的圆建立累加器空间，边缘点依据梯度方向对可能的圆心位置进行投票。累加器空间中投票数超过阈值的位置被认为是圆心候选点，只需要根据预先设置的参数筛选出局部最大值点作为最终的圆心就可以获得图中的圆。

三、系统设计与实现

注：本系统环境为 Apple M3 macOS Sequoia 15.0.1，python 与 C++环境见 coda/README.md 文件。

1. 系统总体设计

本系统的 Canny 算法使用 C++实现，使用 pybinding 包装成为 Python 接口，与 python 实现的 HoughCircles 算法一同存放在 mycv.py 中。系统主实验框架实现在 coin_detection.py，可以轻松地在不同实现之间切换。这种设计能够方便地添加新的检测策略，对比不同算法性能。参数调优 WebAPP 的 falsk 服务器单独实现在 app.py 中，并使用 template/index.html 作为 web 前端。总体架构设计见下图：



2. Canny 算法模块实现 (C++实现)

Canny 算法主要实现在 canny.cpp 文件中，这部分包含一个函数 myCanny，其参数包括：

参数	类型	说明
src	const cv::Mat&	输入图像，支持 CV_8U 或 CV_16S 类型
dst	cv::Mat&	输出的边缘图像，二值图像，边缘像素值为 255
lowThreshold	Double	低阈值，用于滞后阈值处理，取值范围[0, 255]
highThreshold	double	高阈值，用于滞后阈值处理，取值范围[0, 255]
apertureSize	Int	Sobel 算子的大小，可选值为 3、5 或 7
L2gradient	bool	是否使用 L2 范数计算梯度幅值

自行实现的 Canny 算法为了优化性能，采用了多种优化方式，主要策略如下：

2.1 OpenMP 并行计算优化

```
#pragma omp parallel for
for (int i = 0; i < rows; i++) {
    // 并行处理每一行
}
```

通过 OpenMP 实现了图像行级别的并行处理，充分利用多核 CPU 资源，显著提高了算法在大尺寸图像上的性能。

2.2 内存访问优化

```
// 创建并填充 border 以简化后续处理
cv::Mat mag(rows + 2, cols + 2, CV_32F);
mag.setTo(0);
```

通过增加图像边界、合理组织数据结构，减少边界条件判断，优化了内存访问模式，提高了缓存命中率。

2.3 定点数表示与分支优化

```
const int TG22 = 13573; // tan(22.5 度) * 2^16
```

使用定点数表示角度值，避免了频繁的浮点运算。同时，算法中对分支条件进行了重组和简化，减少了 CPU 分支预测失败的可能性。

3. HoughCircles 算法模块实现

HoughCircles 算法主要实现在 mycv.py 文件中，这部分包含函数 HoughCircles 的实现，其参数包括：

参数	说明
Image	输入图像，灰度图像
method	检测方法，目前仅支持 cv2.HOUGH_GRADIENT
dp	累加器分辨率与图像分辨率的反比，dp=1 时累加器与输入图像具有相同分辨率
minDist	检测到的圆之间的最小距离
param1	Canny 边缘检测的高阈值，低阈值为高阈值的一半

param2	累加器阈值，值越小检测到的圆越多
minRadius	最小圆半径
maxRadius	最大圆半径

自行实现的 HoughCircles 算法的关键实现细节包括：

1. **基于梯度方向的投票**：利用边缘点的梯度方向信息，将投票限制在梯度方向上，提高了检测准确性。

```
# 计算每个边缘点的梯度方向
sobelx = cv2.Sobel(blurred, cv2.CV_32F, 1, 0)
sobely = cv2.Sobel(blurred, cv2.CV_32F, 0, 1)
magnitude = np.sqrt(sobelx ** 2 + sobely ** 2)
direction = np.arctan2(sobely, sobelx)

# 根据梯度方向计算潜在圆心
theta = direction[y, x]
center_x = int(x - r * np.cos(theta))
center_y = int(y - r * np.sin(theta))
```

2. **多半径搜索**：对每个可能的半径值创建专用的累加器空间，避免了半径维度上的精度损失。

```
# 为每个半径创建单独的累加器数组
accumulators = {r: np.zeros((acc_height, acc_width), dtype=np.int32)
                 for r in range(minRadius, maxRadius + 1)}

# 对每个半径单独进行投票和处理
for r in range(minRadius, maxRadius + 1):
    acc = accumulators[r]
    centers = np.argwhere(acc > param2)
    for center_y, center_x in centers:
        x = int(center_x * dp)
        y = int(center_y * dp)
        accumulator_value = acc[center_y, center_x]
        candidates.append((x, y, r, accumulator_value))
```

3. **非极大值抑制**：对检测到的圆进行筛选，确保相近的圆被合理合并。

```
# 按照累加器值降序排列候选圆
candidates.sort(key=lambda c: c[3], reverse=True)

# 非极大值抑制
circles = []
for x, y, r, _ in candidates:
    too_close = False
    for cx, cy, cr in circles:
        if np.sqrt((x - cx) ** 2 + (y - cy) ** 2) < minDist:
            too_close = True
            break
    if not too_close:
        circles.append([x, y, r])
```

4. 参数调优 Web APP 实现

为了便于调试和参数优化，开发了一个基于 Flask 的 Web 应用，该 Web 应用具有以下特性：

1. **实时参数调整**：提供了直观的滑动条界面，可实时调整 Canny 和 Hough 变换的各项参数。
2. **结果可视化**：同时展示原始图像、边缘检测结果和圆形检测结果，便于直观比较。
3. **性能分析**：显示各处理步骤的耗时统计，便于性能分析。

四、系统性能与效果

1. Canny 算法性能

通过 benchmark 测试，对比了自实现 Canny 与 OpenCV 实现的性能差异（100 次实验的平均性能，本处为 canny_cpp/main.cpp 的输出结果），实验结果表面，自主实现的 canny 算法已经较为接近 opencv 的实现性能了：

```
性能比较结果：
-----
优化 Canny:
平均时间: 23.7293 毫秒
最小时间: 21.4348 毫秒
最大时间: 58.4571 毫秒

OpenCV Canny:
平均时间: 3.52569 毫秒
最小时间: 2.99096 毫秒
最大时间: 5.04992 毫秒

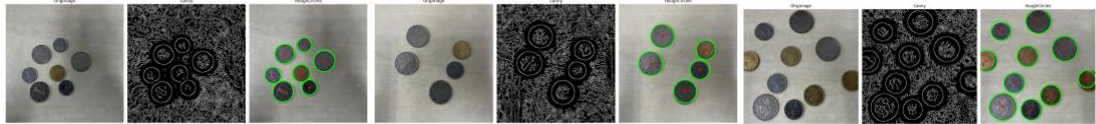
相对性能:
优化实现比OpenCV慢 20.2037 ms

结果差异:
与OpenCV的平均像素差异: 0
```

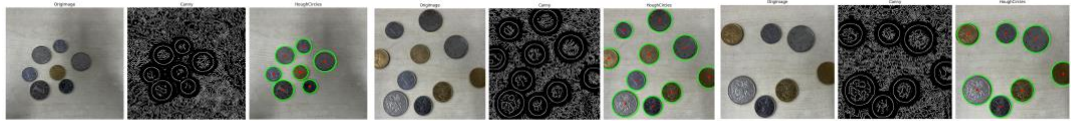
2. HoughCircles 算法性能

共进行 18 组对比实验，效果可见 output_mycv 下的图片与 json 报告。由于篇幅限制，本处仅给出 3 组实验的对比效果，自实现的 HoughCircles 算法与 opencv 的实现对比表现如下，：

MyHoughCircles:



OpenCVHoughCircles:



总体而言，自实现算法在检测效果上已经贴近 OpenCV 原生实现，受限于 python 实现，在速度性能上稍有差距。

3. Web APP 效果

Web 应用在参数调优上发挥了重要作用。通过实时调整参数并观察效果，发现 Canny 的两个阈值与 Hough 变换的累加器阈值对结果影响最大。基于多次测试，确定了不同场景下的最优参数区间，在我使用的图片下，硬币检测的推荐参数见下图。

硬币检测参数调优

上传图像

选择文件 1.jpg

上传

参数设置

参数预设

默认设置

高灵敏度

低噪声

大硬币

Canny边缘检测

阈值1: 10

阈值2: 30

孔径大小: 5

☒ L2梯度

Hough圆形检测

DP (累加器分辨率): 1

圆之间的最小距离: 30

参数1 (Canny高阈值): 30

参数2 (累加器阈值): 24

最小半径: 10

最大半径: 100

处理图像

保存参数

结果

原始图像

边缘检测

圆形检测

检测到的硬币数: 7
Canny耗时: 2.61 毫秒
Hough耗时: 2.15 毫秒
总耗时: 10.69 毫秒